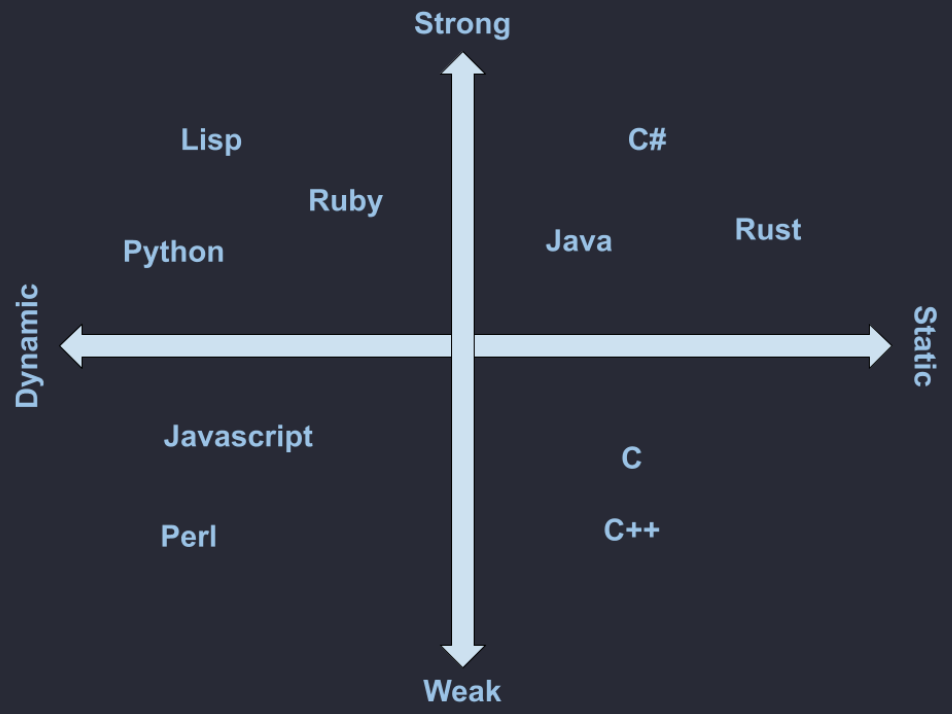
Type System Nuances

CS-3180

So far

- Properties of type system
- Strong and Weak
- Static and Dynamic
- "Two Axis" System



Nuances

- These things exist on a spectrum
- We should spend some time discussing what lies in between
- Get a feel for a few different languages

Move up the scale

- The + operation is well defined for different types
- Under the surface, this shouldn't really work
- Most just let you concatenate or convert
- But not all
- Call a language's ability to prevent type errors: Type Safety

Javascript

- Weak, Dynamic typing

```
var integerCount = 5;
var decimalWeight = 2.5;

// silently coercion the integer to
// float and adds them
var total = integerCount + decimalWeight;

console.log(total);
```

- Not type-safe

Java

- Strong, static typing

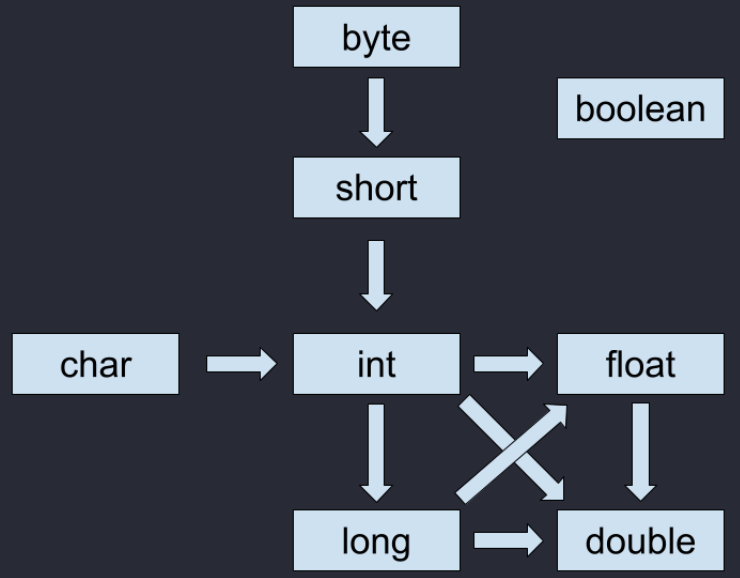
```
public class Main {
    public static void main(String[] args) {
        int integerCount = 5;
        double decimalWeight = 2.5;

        // implicit changes integerCount into 5.0 (at runtime)
        double total = integerCount + decimalWeight;

        System.out.println(total);
    }
}
```

Java

- Only allowed because no information is lost
- Type has "widened"
- Prevents type narrowing, without an explicit cast
- Mostly type safe?
- (bools are unique, ignore online charts)



Rust

- Strong, Static typing

```
// warning: this code will not compile
fn main() {
    let integer_count: i32 = 5;
    let decimal_weight: f64 = 2.5;

    // type mismatch, Rust will refuse the implicit cast
    // even though no "information" is lost
    let total: f64 = integer_count + decimal_weight;
}
```

- We would need some explicit conversion here

Spectrum

- So even when languages fall in the same quadrant
- Some are still more type-safe than others
- Systems will scale better
- With more type safe code

Type Inference

- Anatomy of a variable
- Most variables, that is
- (Without any sub-typing)
- (In statically typed languages)

```
[TYPE] [NAME] = [DATA]
```

Type Inference

- If I ommitted these types
- You could likely figure out what these are, right?

```
[?] ten = 10;
[?] s = "Hello World"
```

Type Inference

- Often, the compiler is smart enough
- To infer these types as well
- Call this behavior "Type Inference"
- Common in statically typed languages
- Java -> var keyword
- C++? -> auto keyword
- Rust -> Omit the type

```
var ten = 10;
var s = "Hello World";
```

Type() Functions

- Dynamically Typed Languages -> Types exist at run-time
- Which means that types are usually some kind of value
- Most languages in this boat provide a way of checking the type
- For the developer to see

Type() Functions

```
print(type(42))
print(type("Hello"))
print(type([1, 2, 3]))
```

- class 'type'
- Types are VALUES in these languages

Type() Functions

- This is often a problem if you want to enforce a type in your code

```
def square(value):
    # introduce "not" and "in"
    if type(value) not in (int, float):
        return "Error: please give me a numeric type"

    return value ** 2
```

- Notice: return types are different
- Can affect the callers

Codepoint

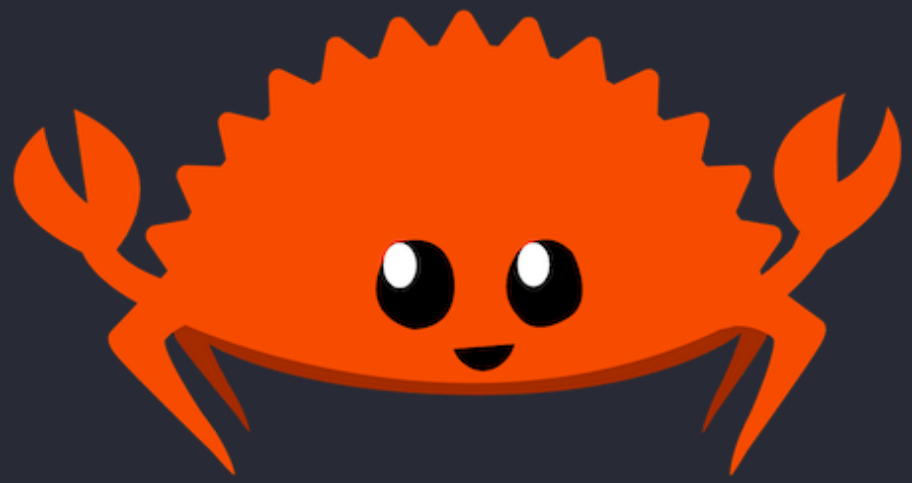
- This is especially a problem for libraries
- Code that is written once, under many different contexts
- Library devs do *not* control who calls their code
- Riddled with type checks everywhere
- Let's take a peek at a library I maintain: github.com/oseda-dev/codepoint

People hate this

- Dynamic types work for quick scripting
- But industrial scale systems
- Riddled with bugs if they are dynamically typed
- What choices do developers have?

Only use type-safe languages

- Idyllic solution
- Almost never possible
- "Re-write it in Rust!"



Type Hints

- Python added a system to denote types in your code
- *Added in 3.5*
- Optional System
- Uses : and ->

```
def greet(user: str) -> str:
    return f"Hello, {user}!"
```

Type Hints

- Not everybody will use them
- Python *ignores* them
- LSP might warn you? maybe?

```python
def greet(name: str) -> str:
    # we can just lie
    return 12345

# we can pass whatever we want actually
result = greet(42)
```

TypeScript

- Javascript's solution to this is Typescript
- Typescript = Javascript-like language, with *static* typing
- `tsc` compiler
- "Compiles" into Javascript?

TypeScript

```
function greet(name: string) {
    return "Hello, " + name;
}
// this will not compile
greet(42);
```

- Maybe a better option than type hints
- But...

Any Type!

```
let dynamicValue: any = "Hello World";

// re-assign this to whatever you want! yay!
dynamicValue = 42; // number type
dynamicValue = true; // boolean type
dynamicValue = { id: 1 }; // object type (!!!)
```

Perils of JS

- JS is one of the most dynamic weakly typed languages
- That is used at such a grand scale
- Comes with a lot of caveats
- Let's have a look at them
- https://jsisweird.com/

Questions?